Knowing When to Respond: Instruction-Conditioned Streaming Detection for Proactive Audio Understanding; Checkpoint II

Bailey Malota
Computer Science
Virginia Tech
Springfield, Virginia
baileym04@vt.edu

Jaemyung Yu
Computer Science
Virginia Tech
Arlington, Virginia
jaemyungy25@vt.edu

Martin Balla
Computer Science
Virginia Tech
Arlington, Virginia
martinb04@vt.edu

Maximilian Ilyev
Computer Science
Virginia Tech
Sterling, Virginia
maximilianilyev@vt.edu

Abstract
Audio-aware assistants must decide not only what they hear but when a response would help. We are building a system for the audio-only subtrack of Track II of the HEARTBEAT Challenge [1], "Streaming Proactive Understanding." A system receives one written instruction and then one to two minutes of streaming speech, sound, or music. Using only the audio heard so far, it must respond with timed free text each time the instruction's condition is met, and stay silent otherwise. After the organizers redefined the track and released 29,371 training records, we rebuilt our Checkpoint 1 system around the real task. A frozen causal BEATs encoder and causally exposed Whisper word timings feed a small GRU whose features are modulated by the instruction through FiLM. The GRU predicts, every 0.5 s, whether a response is due. A calibrated threshold decoder decides when to speak, and a separate writer produces the text. On a 1,200-record training subset, speech features roughly double joint F1. Grouped calibration with per-instruction-type thresholds reaches 0.123 on 300 development records, against 0.033 for a fixed-time prior. Instructions about speech that quote no exact phrase remain unsolved. This report describes the system as built, how it changed since Checkpoint 1, each member's progress, and our plan to the final submission.

I. Introduction

Smart speakers, home-monitoring devices, and assistive technologies increasingly listen continuously to their surroundings. Their value depends on noticing the moments that matter, such as a smoke alarm, a knock at the door, or a person asking for help, while staying quiet the rest of the time. Both failure modes are costly. A missed event can be dangerous, while an assistant that interrupts over every sound quickly becomes an annoyance that users switch off. Modern audio models are good at recognizing what a sound is, but recognition alone does not tell a system whether, when, or how to respond.

The HEARTBEAT Challenge [1] formalizes this gap. Our proposal and Checkpoint 1 targeted an early version of Track II, in which a system heard 10–90 s of non-speech audio and chose one of four actions. On October 9 the organizers redefined the track as "Streaming Proactive Understanding," and on October 27 they released its data. A system now receives one instruction at the start of a stream, for example "Alert the first time someone knocks on the door and immediately asks a question from outside, quoting what was asked." It then processes 60–120 s of audio chunk by chunk. Each time the condition is met, it must respond with free text, timestamped at the end of the chunk that produced it. It may use only audio already received, must log every chunk, and is scored by precision, recall, and a joint F1 that penalizes misses, false activations, and duplicates.

The released data differs from what we planned for in three ways that shaped this checkpoint. First, the audio is not mostly environmental sound: 28% of recordings are speech, 34% are mixed, and 44% of instructions mention speech. Second, responses are free text that the instruction itself often dictates, not actions. Third, every record has at least one response, so silence is never free.

This problem is well suited to deep learning. It needs learned acoustic representations, temporal modeling of how a stream unfolds, conditioning on a natural-language instruction, and text generation grounded in audio. Our staged goals for the semester and their status are:

1) Build an end-to-end pipeline and internal evaluation on the released data, and compare with the organizer baseline. *Done on a 1,500-record subset.* The organizers released a downloader but no baseline model or scorer, so we compare against a fixed-time prior and a never-respond system.
2) Deliver a complete end-to-end system with template responses (Checkpoint 2). *Done:* the `rule` writer (Section IV).
3) Replace templates with a fine-tuned audio language model and quantify the gain. *Implemented, not yet evaluated.*
4) Submit a valid container to the challenge. *Pending;* the final container is due December 23.

Our contributions are (i) a decoupled detect-then-write streaming system (Listing 1), in which the instruction conditions a small causal detector through FiLM and a separate writer produces text only when the detector fires; (ii) causal speech evidence: speech-recognition word timings that become visible only once a word has ended, matched against the instruction, which roughly doubles joint F1; (iii) calibration for a streaming metric, using grouped out-of-fold threshold search with per-instruction-type thresholds; and (iv) a con-

trolled comparison of response writers on identical response times.

II. Related Work

A. Audio Representation and Event Detection

Large-scale weakly labeled data such as AudioSet [2] enabled general-purpose audio encoders, including CNN-based PANNs [3], the Audio Spectrogram Transformer [4], and self-supervised BEATs [5]. CLAP [6] aligns audio with text in a shared embedding space. Sound event detection, advanced largely through the DCASE challenges [7], localizes events in time. These models answer what happened and when. We use BEATs as a frozen feature extractor.

B. Speech Recognition and Keyword Spotting

Whisper [8] transcribes speech robustly across domains and gives word-level timestamps, which we use as a feature source rather than as an output. Keyword spotting [9] detects a fixed phrase in a continuous stream with a small model. Many of our instructions are an open-vocabulary form of this problem: the phrase to listen for arrives in the instruction, so it cannot be trained in advance.

C. Audio-Language Models

Audio captioning datasets such as AudioCaps [10] and Clotho [11] link audio to language. Audio-language models connect an audio encoder to a large language model: Pengi [12], LTU [13], SALMONN [14], and Qwen2-Audio [15]. They can describe and reason about audio, but they answer when asked and do not decide whether an unprompted response is due. We use Qwen2-Audio, adapted with LoRA [16], as one of our response writers, and leave the decision of when to respond to a much smaller model.

D. Turn-Taking, Proactivity, and Online Detection

Deciding when to speak has been studied for conversation, from turn-taking models [17] to voice activity projection [18], and for proactive dialogue [19] and agents [20] that operate on text or user activity. Online action detection [21] labels each video frame as it arrives, without seeing the future. It is the closest analogue to our per-chunk decision. To condition that decision on an instruction, we use feature-wise linear modulation (FiLM) [22], which scales and shifts intermediate features as a function of a conditioning input.

III. Task, Data, and Evaluation

A. Task and Protocol

Each audio-only record holds one instruction, asked at $t = 0$, a 16 kHz mono recording, and one or more reference responses with their times. Example: "Alert the first time someone knocks on the door and immediately asks a question from outside, quoting what was asked." The reference response is "How you doing in there?" at 68.2 s. Records are tagged with a level (L1–L3) and an audio type; a task-type field is always empty. The system must act online, log every chunk (empty when silent), and timestamp each response at the end of its chunk, with no backdating.

TABLE I
Track II audio-only data, measured from record metadata.

	Train	Dev
Records	29,371	1,200
Audio (one copy per record)	866 h	36 h
Packed download	84.1 GB	3.5 GB
Duration, median (range)	112 s (60–120)	113 s (62–120)
Level L1 / L2 / L3	46 / 40 / 14%	46 / 40 / 14%
Mixed / sound / speech / music	34 / 30 / 28 / 8%	34 / 30 / 28 / 8%
Responses per record 1 / 2 / 3+	89 / 7 / 5%	88 / 7 / 5%
Instruction mentions speech	44%	48%

Table I summarizes the data. Several records ask different questions about the same recording (16 distinct recordings in the first 20 dev records), so we cache each distinct recording once, keyed by a hash of its audio, and keep records that share a recording in the same cross-validation fold.

B. Evaluation

The organizers publish the metric family (precision, recall, joint F1) and that misses, false activations, and duplicates all cost, but not yet a scorer, a timing tolerance, or a text criterion. Our scorer makes these explicit and configurable. A prediction pairs with a reference if it falls within $[t_{\text{ref}}, t_{\text{ref}} + 3\text{ s}]$ (Fig. 1); each reference takes the earliest unclaimed prediction, and unpaired predictions, including duplicates, are false positives. Timing F1 counts every pair. Joint F1 counts a pair only if the response's token F1 against the reference is at least 0.3. Scores are micro-averaged over responses, with 95% confidence intervals from a 1,000-sample bootstrap over records. Every run also prints two baselines: *never respond*, which scores 0 because every record has a response, and a *prior* that answers once per record at the training set's median first-response time (44 s) with the `rule` writer's text.

C. What the Redefinition Changed

Checkpoint 1 listed four consequences of the redefinition. All four are now handled:

- *Output protocol:* the system logs one entry per 0.5 s chunk and timestamps each response at its chunk's end. The old action vote, which looked 3 s past the reported time, is gone along with the actions.
- *Multiple responses:* the decoder re-arms after each response (Section IV).
- *Metric:* we score with precision, recall, and joint F1. The old weighted total is retired.
- *Instructions:* the instruction conditions both the detector and the writer. We target the audio-only subtrack.

Pre-release prototyping. Before the release we built the pipeline on a pseudo-challenge set mixed from ESC-50 [23] and on 30 synthetic scenes (Checkpoint 1). Two lessons carried over. First, encode causally: chunked BEATs let attention carry later audio into earlier frames, and causal encoding was worth +0.16 on that set. Second, choose thresholds by cross-validation that holds out whole source groups. The four-action formulation, the learned event probe, and the confirmation

window did not carry over, because the released labels contain no actions or timed event spans.

IV. System Design

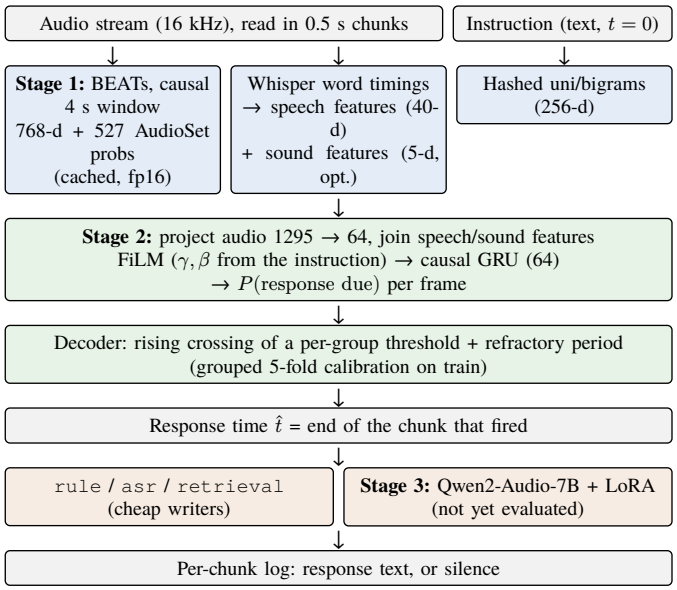
A. Architecture

Our system has three stages (Listing 1). Stages 1 and 2 decide when to respond; Stage 3 decides what to say. Every frame lies on a 0.5 s hop grid, and frame i makes its decision at $0.5i + 1$ s using only audio up to that time.

The system runs in two phases. An extraction phase runs the frozen models once per distinct recording and caches their outputs, keyed by a hash of the audio: BEATs frames, and Whisper words with their start and end times. A decision phase loads the caches, builds the speech and sound features against the record's instruction, runs Stage 2, decodes, and calls the writer once per response. It writes one log entry per 0.5 s chunk, holding the chunk's end time and the response text or an empty string as the challenge requires, plus the detector's score for analysis. In the container, the extraction phase must instead run on the stream as it arrives (Section VI).

B. Stage 1: Frozen Audio Encoder

Each frame is embedded by BEATs (iter3+, fine-tuned on AudioSet) [5] from the 4 s of audio ending at its decision time, which gives a 768-d embedding and 527 AudioSet class probabilities (1,295-d per frame). We re-implemented BEATs for inference and matched the reference code to 10^{-6}. The transformer runs in fp16 on GPU (embeddings agree with fp32 to cosine ≥ 0.99999), and outputs are cached in fp16, about 650 KB per recording, or 15–19 GB for all of train. Tests overwrite the audio after a frame's decision time and require earlier frames to stay unchanged.



Listing 1. Pipeline as built. The instruction conditions Stage 2 through FiLM, is matched against the transcript and AudioSet class names to form the speech and sound features, and is read by every writer. Only Stage 2 and the decoder are trained on the challenge data; Stage 1 and Whisper are frozen.

Change from Checkpoint 1: the learned event probe is gone, since the official labels have no event spans to train it on.

C. Instruction and Speech Features

Instruction. The instruction is encoded as hashed unigrams and bigrams in a 256-d, log-scaled, L2-normalized vector. A MiniLM sentence encoder [24], [25] is a switchable alternative.

Speech features. BEATs hears that someone is speaking, never what is said, yet 44% of instructions turn on spoken words. We transcribe each recording once with Whisper base.en [8] through faster-whisper [26], with Silero voice-activity detection [27] so that music and noise are not transcribed into words. We turn the word timings into 40 features per frame (Table II). A word becomes visible to a frame only once its end time has passed. Generic instruction words such as "notify," "someone," and "says" are ignored when matching, since every instruction uses them. Quoted phrases are matched in order but may skip a couple of intervening words, so a phrase still matches when Whisper inserts a filler word.

Sound features (optional). The instruction's condition is matched to its eight closest AudioSet class names, by stemmed word overlap or MiniLM similarity. Five per-frame features summarize those classes' probabilities: best match, weighted maximum (linear and log), and its rise over 2 s.

D. Stage 2: Instruction-Conditioned Causal Detector

The audio features are standardized and projected from 1,295 to 64 dimensions with a ReLU. The speech (and sound) features skip this projection, so that 40 features are not drowned out, and are concatenated with it. FiLM [22] then modulates the result as $h \cdot (1 + \gamma) + \beta$, where γ and β are linear in the instruction vector. A one-layer unidirectional GRU [28] with 64 hidden units reads the sequence. A linear head, which also sees a 32-d projection of the instruction, outputs per frame the probability that a response is due. The model has about 177k parameters.

Why FiLM. A unit test builds synthetic clips that each contain a bell and a knock, and the instruction names one of

TABLE II
The 40 speech features at each frame, computed only from Whisper words whose end time has passed.

Feature	Value at a frame
New words	Words ending in the last 0.5 s, divided by 4 (capped at 1)
Speaking	1 if any word ended in the last 2 s
Confidence	Mean Whisper probability of the last 2 s of words
Instruction overlap	Share of the last 2 s of content words that occur in the instruction
Instruction word	1 if a word ending in the last 0.5 s occurs in the instruction
Phrase coverage	Best in-order coverage of a quoted instruction phrase by the last 2 s of words
Phrase completed	1 if a quoted phrase was completed in the last 0.5 s
Phrase count	$\log(1 + n)$ for n phrases completed so far, for "each time" instructions
Word bag (32-d)	Hashed bag of the words in the last 3 s, for triggers about meaning rather than wording

them. Appending the instruction vector to every frame failed this test: the model answered both events (precision 0.54). FiLM, which lets the instruction turn relevant features up and others down, passes on every seed tried.

Targets and loss. A frame is positive if its decision time falls within 2 s after a reference response time (four frames), and never before it. Positives are about 2% of frames. We use binary cross-entropy with the positive weight set to the negative-to-positive ratio, capped at 20, since full parity made every frame look like a trigger. Training uses Adam (learning rate 3×10^{-3}, weight decay 10^{-4}), input dropout 0.1, batches of 16 records, and 15 epochs, on CPU or GPU.

Change from Checkpoint 1: one binary output replaces the four-action softmax, and the instruction now conditions the detector.

E. Decoding and Calibration

The decoder fires when the probability rises through a threshold (Fig. 1). It re-arms once the probability falls below the threshold, and enforces a refractory period after each response, so a stream can receive several responses but not two for one event. Thresholds (12 values from 0.2 to 0.99) and refractory periods (2, 5, or 10 s) are chosen on the training set only. We fit $K = 5$ models, each scoring the fifth of train it did not see, with folds grouped by recording, then search the grid on all out-of-fold predictions at once and refit on all of train ($K + 1$ fits).

We compare four decode policies on the same out-of-fold probabilities: one global setting; a setting per instruction group (instruction quotes a phrase, mentions speech, or neither), found by coordinate ascent on overall joint F1; and each with an optional fallback that lowers the bar once if nothing has fired by 30–60 s. The default is the grouped policy without fallback (Section V).

F. Stage 3: Response Writers

Writers receive the instruction, the response time, and the transcript, and share one interface:

- `rule`: the reply the instruction dictates (for example, a phrase it asks the system to say), otherwise the instruction's condition restated. The dictated reply covers only 179 of 29,371 training instructions, but matches its reference exactly in 169 of them.

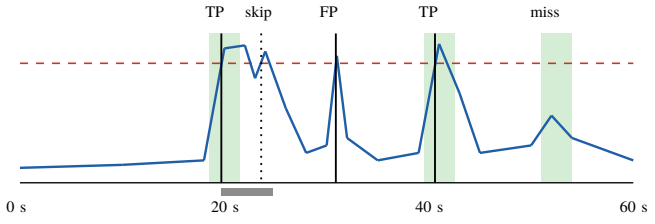


Fig. 1. Decoding and scoring on one stream (schematic, illustrative values). A response is emitted when the detector's score (blue) rises through the threshold (dashed); no new response is allowed during the refractory period (gray bar), so the second crossing (dotted) is skipped. A response is a true positive if it lands in a reference's window $[t_{\text{ref}}, t_{\text{ref}} + 3\text{ s}]$ (green). The response at 31 s is a false positive, and the third reference, whose score never crosses, is a miss.

- `asr`: as `rule`, but quotes the last 6 s of transcript when the instruction asks for the words that were said.
- `retrieval`: the first response of the most similar training instruction (TF-IDF cosine).
- `audiolm`: Qwen2-Audio-7B-Instruct [15], given the instruction, the last 20 s of audio ending at the response time, the Whisper words in that window, and the record's earlier responses. It decodes greedily up to 48 tokens and falls back to `rule` when it breaks format. Fallbacks are counted, not hidden. A LoRA adapter [16] (rank 16, attention and MLP projections, audio encoder frozen) is trained with response times jittered 0–3 s late, so the writer learns to cope with imperfect timing from Stage 2.

Calibration always uses the cheap `rule` writer. The audio LM rewrites only the final policy's responses, so writers are compared on identical response times.

Change from Checkpoint 1: the Qwen3.5 text-only generator is replaced by an audio LM that hears the audio itself, because responses depend on what was said and heard, not on a list of detected events.

G. Design Choices

Detect, then write. A single audio LM could in principle listen to the stream and decide both when and what to say, but it would have to run at every chunk. A median record of 112 s holds about 220 decisions at our 0.5 s hop, so the 1,200 dev records alone would need over a quarter of a million calls to a 7B model, each over up to 30 s of audio. Our detector makes those decisions from cached features with about 177k parameters, and the writer runs only when the detector fires, about once per record (355 responses for 300 records). Separating the two also lets us train and calibrate the detector many times, since grouped K-fold calibration needs $K + 1$ fits per run. And audio LMs are trained to answer when asked, not to stay silent.

Frozen, cached features. Encoding the training set causally takes roughly a week of CPU time (0.2 s per audio second) or about 13 h on one consumer GPU in fp16. Encoding once and caching makes a Stage 2 run take minutes (about 7 min on CPU with one held-out split, 20 min with five folds), which made the ablations in Section V affordable. Fine-tuning the encoder would repeat the encoding cost every epoch.

One binary output. The released labels contain response times and texts but no actions, and every record has at least one response. The detector therefore answers one question per frame, whether a response is due now, and leaves everything about what to say to the writer.

Hashed instruction features. Hashed n-grams keep the exact words of the instruction, which quoted-phrase triggers depend on. They cost nothing to compute and need no model download inside the container. A MiniLM sentence encoder is available as a switch but has not yet been compared on the detector.

Speech evidence in the detector, not only the writer. Giving the transcript only to the writer would not help the system respond at the right time, because the detector must hear the words to know when the condition is met. Speech features put

that evidence into every frame, and they produced the largest gain we measured.

H. Languages, Tools, and Compute

The system is written in Python with PyTorch, Hugging Face Transformers, and PEFT. It runs in a shared dev container (CPU and AMD ROCm variants) with locked dependencies, and the code is on GitHub. About 200 unit tests include the causality checks for the encoder, the speech and sound features, and the audio LM's input window. The official data is read straight from the organizers' packs by HTTP range requests, so no second copy is exported.

Full-scale work runs through Slurm on Virginia Tech ARC (Falcon, NVIDIA A30). Both data packs are in group storage, and the BEATs and Whisper caches for dev are built. Stage 2 trains on the GPU; prediction is batched 64 records at a time, and CPU results are bit-identical to the earlier CPU-only code. Measured throughput: BEATs fp16 takes 0.015 s per audio second on a consumer GPU (36 h of dev audio in about 30 min). Whisper takes 0.46 s per recording on an A30 against 0.93 s on four CPU cores, with near-identical transcripts (8 dev recordings, 898 s of audio). Transcribing all of train is therefore an estimated 3.5–4 h on one A30. Final systems are submitted as containers, so model size and latency matter for the writer choice.

I. Data Sets

We train and calibrate only on the Track II audio-only training split and report results on its development split (Table I). The frozen components bring their own training data. BEATs was pre-trained and fine-tuned on AudioSet [2], about two million 10 s clips, whose 527 class names also define our sound features. Whisper was trained on 680,000 hours of weakly supervised web audio [8]. The optional MiniLM encoder (all-MiniLM-L6-v2) was trained on sentence pairs [24], [25], and Qwen2-Audio on large-scale audio–text data [15]. Before the release we built a pseudo-challenge set from ESC-50 [23], 2,000 five-second environmental clips in 50 classes (Section III).

V. Results

All results use the first 1,200 training records (4% of train) and the first 300 dev records (362 reference responses) in manifest order, with seeds 0, 1, and 2 and the `rule` writer unless noted. They measure design choices on a subset under our own scorer, not leaderboard performance. A single run's 95% interval is about ± 0.03 joint F1; for example, PANNs with speech features on seed 0 scores 0.098 [0.067, 0.131].

Speech features matter most. Adding them roughly doubles joint F1 with either encoder (Table III), with non-overlapping intervals, and lifts the model clearly above the prior. Without them the model barely beats a system that knows nothing about the audio. The gain is where it should be (Table IV): instructions that quote a phrase go from almost nothing (0.010) to the best group (0.155), and timing F1 on them rises from 0.046 to 0.327. Speech features also stop over-firing:

TABLE III

Dev joint F1, mean (range) over 3 seeds, and mean timing F1. "1 split": threshold from one 20% held-out split; "5 folds": grouped K-fold calibration. †: run in a different CPU environment, where the same code and seed can differ by up to 0.012.

System	Calibration	Joint F1	Timing
Never respond	n/a	0	0
Prior at 44 s	n/a	0.033	0.054
PANNs	1 split	0.047 (0.037–0.058)	0.080
BEATs causal	1 split	0.055 (0.050–0.062)	0.080
PANNs + speech	1 split	0.099 (0.090–0.108)	0.186
BEATs + speech	1 split	0.109 (0.099–0.124)	0.201
BEATs + speech	5 folds	0.114 (0.103–0.128)	
BEATs + speech†	5 folds, global	0.118 (0.098–0.136)	
BEATs + speech†	**5 folds, grouped**	**0.123 (0.093–0.142)**	

TABLE IV

Dev joint F1 by group, without → with speech features (PANNs, seed 0, single split). With BEATs, quoted-phrase instructions reach 0.189 and speech audio 0.138.

Group	Records	Without	With
Instruction quotes a phrase	72	0.010	**0.155**
No quoted phrase	228	0.050	0.072
Audio type: speech	132	0.018	0.108
Audio type: mixed	58	0.020	0.102
Audio type: sound	75	0.082	0.090
Audio type: music	35	0.081	0.058
Level L1 / L2 / L3	166 / 94 / 40	0.042 / 0.022 / 0.051	0.128 / 0.050 / 0.062

predictions fall from 2.4 times the number of references (873 for 362) to about one (355).

The causal encoder loses nothing. PANNs frames see a little future audio; causal BEATs frames see none, yet score the same or slightly better on every row. We use causal BEATs from here on.

Grouped K-fold calibration makes the held-out estimate honest (Table V). A single 20% split scores 36 settings on about 240 records and rewards luck: its held-out joint F1 overstated dev by 0.036 on average (0.145 vs. 0.109). With five folds the same settings are scored out-of-fold on all 1,200 records, and the estimate lands within 0.002 of dev (0.115 vs. 0.114). Without speech features, five folds also stabilize the choice: the dev seed spread falls from 0.012 to 0.004, and silent records no longer swing between 18 and 126. With speech features, even out-of-fold calibration picks thresholds of 0.9–0.97, and 130–175 of 300 dev records receive no response. This silence is real rather than calibration noise. Speech features give sharp evidence for quoted-phrase instructions and little for the rest, and under joint F1 a single threshold for both is best set high.

Per-group thresholds help a little; fallbacks do not. Paired over ten runs, grouped minus global averages +0.007, never worse than −0.005 and up to +0.025 (Table VI). The fallback removes up to 55 silent records, but the forced answers mostly

TABLE V

ONE HELD-OUT 20% SPLIT VS. GROUPED 5-FOLD CALIBRATION (BEATS + SPEECH FEATURES). "HELD-OUT" IS THE JOINT F1 THAT CALIBRATION ITSELF ESTIMATES ON TRAIN, "DEV" THE MEASURED DEV SCORE, "SILENT" THE DEV RECORDS WITH NO RESPONSE (OF 300).

Calibration	Seed	Held-out	Dev	Threshold	Silent
1 split	0	0.154	0.099	0.95	170
	1	0.151	0.103	0.95	174
	2	0.129	0.124	0.90	89
5 folds	0	0.117	0.110	0.90	130
	1	0.119	0.103	0.95	174
	2	0.110	0.128	0.97	175

TABLE VI

DEV JOINT F1 (MEAN OVER 3 SEEDS, 5 FOLDS, BEATS + SPEECH) BY DECODE POLICY AND SOUND FEATURES.

Sound features	Global	Grouped	Grouped + fallback
Off	0.118	**0.123**	0.116
Lexical	0.115	0.120	0.116
MiniLM	0.103	0.111	0.110

miss: it lowers joint F1 in 7 of the 8 runs where calibration chose it (mean −0.008). The global fallback policy never differed from global.

Sound features do not help yet. Lexical matching lifts sound instructions from 0.095 to 0.111 (141 records) but loses as much elsewhere. MiniLM matching is lower and the most seed-sensitive, because long compound instructions dilute the sentence embedding.

Where the errors are. Splitting dev by the instruction groups the decoder uses (Table VII) shows where the score comes from. Quoted-phrase instructions, where the speech features can match the exact wording, reach 0.20–0.22. Sound instructions sit near 0.1, and when sound features are on, calibration gives them lower thresholds (0.7–0.8, against 0.95–0.99 for speech). The main gap is speech instructions without a quoted phrase, such as "tell me when someone mentions dinner." On these 87 dev records joint F1 is 0.00–0.06 in every run, and 53–94% of them get no response. The speech features match only quoted wording, and the hashed bag of recent words has not been enough to recognize a topic. These records are 29% of the dev records we score, so this group is where the largest gain remains.

Writers. On matched pairs, `rule` reaches token F1 0.34 against 0.20 for `retrieval`, and `asr` adds only +0.002 joint F1, because about 1% of instructions ask for the words themselves. The audio LM has not been run yet.

TABLE VII

INSTRUCTION GROUPS ON THE 300 DEV RECORDS AND THEIR JOINT F1 (BEATS + SPEECH FEATURES, 5 FOLDS; QUOTED AND SPEECH: RANGE OVER RUNS, SOUND: MEAN OVER SEEDS). GROUPS ARE ASSIGNED FROM THE INSTRUCTION TEXT ALONE.

Group	Rule	Records	Joint F1
Quoted	Quotes a phrase in quotation marks	72	0.20–0.22
Speech	Mentions saying, speaking, talking, asking, or a word, with no quote	87	0.00–0.06
Sound	Everything else	141	0.095

VI. LIMITATIONS AND NEXT STEPS

- *Scale.* Every number above uses 4% of train. The next step is the full 29,371-record run on ARC, which needs the training caches (BEATs and Whisper, in progress). Model size, epochs, and early stopping will need retuning at that scale.
- *Unquoted speech instructions.* Stage 2 needs a semantic match between the recent transcript and the instruction's condition, for example sentence embeddings over a sliding transcript window, or the audio LM as a verifier of triggers proposed by Stage 2.
- *Over-silence.* Up to 58% of dev records get no response. Per-group thresholds help a little; better evidence for unquoted speech should let thresholds come down.
- *Text quality.* The audio LM and its LoRA adapter are implemented but untested. They will be compared with `rule` on identical response times.
- *Scorer assumptions.* The 3 s window and the 0.3 text threshold are our guesses. We will switch to the official scorer once the organizers release it, and the leaderboard (from November 16) will show how far our scorer is off.
- *Causality of word identity.* Whisper decodes 30 s windows, so a word's timing is causal in our features but its identity may draw on up to 30 s of later audio. The container must transcribe trailing windows instead, and we will measure the cost.
- *Stage 2 causality test.* Stage 2 is causal by construction (unidirectional GRU, per-frame inputs), but unlike Stage 1 it has no test that truncating the future leaves earlier outputs unchanged. We will add one.
- *Container.* The container has not been built. A 7B audio LM must fit the size and latency limits, which the organizers have not yet published.

Privacy and deployment. An always-on listener that transcribes speech also records bystanders, not only its user. Every model in our pipeline runs locally and nothing is sent to a network service, so transcripts need never leave the device. We use only the organizers' released data and collect no new recordings. Joint F1 weighs a false activation the same as a miss, but in deployment they rarely cost the same: a missed smoke alarm is worse than an extra alert. A deployed assistant would need thresholds set for its use, not for the challenge metric, and per-group thresholds give a natural place to do that.

VII. PLAN TO FINAL SUBMISSION

- *Nov 10–16:* build the training caches on ARC; train and calibrate Stage 2 on the full training set; score on all 1,200 dev records.
- *Nov 16–Dec 2:* first leaderboard submissions (two per day); evaluate the audio-LM writer and LoRA against `rule`; semantic speech matching; streaming transcription; container smoke test. Team and resource declarations freeze December 2.

TABLE VIII
MAIN RISKS TO THE FINAL SUBMISSION AND HOW WE PLAN TO HANDLE THEM.

Risk	Mitigation
Training caches finish late	Partial caches already work (train on the first N records); transcription runs on the CPU in the same job as GPU encoding.
Official scorer differs from ours	The window and text threshold are flags. Recalibrate when the scorer is released, and check against the leaderboard from Nov 16.
Audio LM too slow or large for the container	`rule` stays available as the fallback writer. Compare writers on identical response times before the Dec 2 declaration.
Unquoted speech stays near zero	Semantic matching between transcript and instruction; the audio LM as a verifier of triggers that Stage 2 proposes.
Streaming transcription changes the speech features	Measure trailing-window Whisper on dev before Dec 2, and retrain Stage 2 on streaming features if they differ.
Run-to-run variance hides small effects	Three or more seeds, paired comparisons, and one software environment per comparison.

- *Dec 3–11:* ablations (encoder, speech and sound features, decode policy, writer); course final report due December 11.
- *Dec 12–23:* leaderboard validation (closes December 16); final container December 23.

Table VIII lists the main risks. Open technical questions are whether a stronger detector reduces the need for per-group thresholds, and whether an audio LM that hears the audio can also verify when to respond, not only what to say.

VIII. TEAM PROGRESS REPORT

This section lists what each member contributed since Checkpoint 1 (October 20), what is in progress, the problems met along the way, and what comes next.

A. Maximilian Ilyev

Completed: wrote the official data reader, which streams the Track II packs, caches each distinct recording once, and produced the statistics in Table I. Built the official-task pipeline: the instruction-conditioned detector with FiLM, the multi-response decoder, our scorer, and its baselines. Added Whisper transcription and the speech features, BEATs in fp16 on GPU, grouped K-fold calibration, the decode policies, and the sound features, and ran every experiment in Section V. Implemented the Qwen2-Audio writer, its LoRA training script, and the ARC job scripts.

In progress: running the audio-LM writer and its LoRA tuning on ARC. The writer has unit tests for its causal input window but has not yet run with the real model.

Problems: results depend on the software environment. The same code and seed scored 0.110 in the 12-thread ROCm container and 0.098 in a 4-thread CPU environment, so we now compare runs only within one environment and treat 0.01–0.02 as noise.

Next: first leaderboard submissions and coordinating the container.

B. Jaemyung Yu

Completed: tracked the challenge's changing deadlines and kept the team schedule aligned with them. Added the organizers' downloader to the repository as a pinned submodule with locked dependencies, so every member uses the same version. Arranged the team's ARC compute account and shared project storage, and downloaded both audio-only packs (84.1 GB train, 3.5 GB dev). Validated Stage 1 step by step: checked the caches from a few dev records, including that packs and streaming give byte-identical caches, then ran causal BEATs in fp16 on an A30 and built the full dev BEATs and Whisper caches. Wrote the ARC guide the team uses to set up environments and submit jobs.

In progress: building the training caches (the job requests up to two days on an A30, and only one encoding job per split can run at a time), and measuring throughput on higher-end GPUs to judge whether larger models are feasible.

Problems: project storage was not in the initial allocation, so dev work began on per-cluster scratch space, and the project file system does not support POSIX ACLs, so team access needs a team-only group. The dev caches alone (1,200 records) took about an hour to build, and train is about 24 times larger, so compute time limits the model sizes we can try.

Next: share the training caches, then streaming (trailing-window) transcription for the container and the December 2 declarations.

C. Bailey Malota

Completed: set up the ARC environment and ran the test suite on an A30. Made Stage 2 train and predict on the GPU: device selection that falls back to the CPU with a warning, batched prediction, and per-epoch loss logging, with tests showing that CPU results are unchanged bit for bit. Made Whisper run on the GPU and benchmarked it on ARC (0.46 vs. 0.93 s per recording). Made the ARC job train Stage 2 on the GPU by default.

In progress: preparing Stage 2 training for the full 29,371 records (epoch count and early stopping), which waits on the training caches and group-storage access.

Problems: GPU Whisper first failed because its inference engine (CTranslate2) needs CUDA 12 libraries while our PyTorch build ships CUDA 13; loading ARC's CUDA 12 and cuDNN modules fixed it. The benchmark then showed the GPU gain is only twofold, so the training-cache job keeps transcription on the CPU. The course allocation also has only GPU queues, so CPU-only Stage 2 runs still held a GPU; the job now trains on it by default.

Next: scale Stage 2 to the full training set (epochs, early stopping, model size), add the Stage 2 causality test, and study per-group thresholds at full scale.

D. Martin Balla

Completed: Built a Stage 3 evaluation that scores writers apart from timing. It writes each writer's reply at every reference time in all 1,200 dev records, as a perfect Stage 2 would, and reports the share that pass the 0.3 text threshold. It includes bootstrap intervals over records and breakdowns by

instruction group, level, and audio type, and it needs no audio or GPU. rule passes 0.766, echoing the instruction 0.701, and retrieval 0.619. Reading rule's failures led to rule2, which adds two templates: for instructions that ask for a count, it says which occurrence this is, and for instructions that quote a phrase, it replies with that phrase. It raises the pass rate to 0.844, and on quoted-phrase instruction from 0.55 to 0.827. Most remaining failures ask what was said which no template can answer.

In progress: Helping to setup the ARC environment and downloading models required

Problems: The first rule2 prototypes were tuned against dev, so the design moved to 8,000 training records and dev was scored once. rule2′s count also trusts Stage 2 so one false activation or miss shifts every later count.

Next: Score the audio LM, zero-shot and with Maximilian's LoRA adapter, against rule2 on identical response times before the December 2 declaration, including replies written 0–3 s late. Measure the 7B model's size and per-response latency for the container, with rule2 as the fallback if it does not fit. Combine templates for dictated, counting, and quoted instructions with the audio LM for questions about speech content, and switch to the official text metric once it is released.

References

[1] HEARTBEAT Challenge Organizers, "HEARTBEAT Challenge 2027: Human-Centered Audio Agents." [Online]. Available: https://heartbeat-challenge.github.io/

[2] J. F. Gemmeke *et al.*, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.

[3] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.

[4] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," in *Proc. Interspeech*, 2021, pp. 571–575.

[5] S. Chen *et al.*, "BEATs: Audio pre-training with acoustic tokenizers," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[6] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "CLAP: Learning audio concepts from natural language supervision," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[7] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, "Sound event detection: A tutorial," *IEEE Signal Processing Magazine*, vol. 38, no. 5, pp. 67–83, 2021.

[8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. 2023, pp. 28492–28518.

[9] G. Chen, C. Parada, and G. Heigold, "Small-Footprint Keyword Spotting Using Deep Neural Networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 4087–4091. doi: 10.1109/ICASSP.2014.6854370.

[10] C. D. Kim, B. Kim, H. Lee, and G. Kim, "AudioCaps: Generating captions for audios in the wild," in *Proc. NAACL-HLT*, 2019.

[11] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An audio captioning dataset," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020.

[12] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, "Pengi: An audio language model for audio tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] Y. Gong, H. Luo, A. H. Liu, L. Karlinsky, and J. Glass, "Listen, think, and understand," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[14] C. Tang *et al.*, "SALMONN: Towards generic hearing abilities for large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[15] Y. Chu, J. Xu, Q. Yang, and others, "Qwen2-Audio technical report," *arXiv preprint arXiv:2407.10759*, 2024.

[16] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[17] G. Skantze, "Turn-taking in conversational systems and human-robot interaction: A review," *Computer Speech and Language*, vol. 67, p. 101178, 2021.

[18] E. Ekstedt and G. Skantze, "Voice activity projection: Self-supervised learning of turn-taking events," in *Proc. Interspeech*, 2022.

[19] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, "A survey on proactive dialogue systems: Problems, methods, and prospects," in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2023.

[20] Y. Lu and others, "Proactive agent: Shifting LLM agents from reactive responses to active assistance," *arXiv preprint arXiv:2410.12361*, 2024.

[21] R. De Geest, E. Gavves, A. Ghodrati, Z. Li, C. Snoek, and T. Tuytelaars, "Online Action Detection," in *Computer Vision – ECCV 2016*, in Lecture Notes in Computer Science, vol. 9909. Springer, 2016, pp. 269–284. doi: 10.1007/978-3-319-46454-1_17.

[22] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual Reasoning with a General Conditioning Layer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 3942–3951. doi: 10.1609/aaai.v32i1.11671.

[23] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018.

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[25] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Advances in Neural Information Processing Systems*, 2020.

[26] SYSTRAN, "faster-whisper: Faster Whisper Transcription with CTranslate2." 2023.

[27] Silero Team, "Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detector." 2021.

[28] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

Appendix

Task Assignment

Member	Responsibilities to the final submission
Maximilian Ilyev (Lead)	**Primary, evaluation and delivery:** scorer, experiments, leaderboard submissions, container, coordination, report editing. **Secondary, Stage 3:** audio-LM writer integration.
Jaemyung Yu	**Primary, Stage 1 and data:** ARC storage, training caches (BEATs and Whisper), streaming transcription for the container. **Secondary, Stage 2:** sound features.
Bailey Malota	**Primary, Stage 2:** full-scale detector training on ARC GPUs, model size and early stopping, calibration and per-group thresholds, causality tests. **Secondary, Stage 1:** GPU transcription.
Martin Balla	**Primary, Stage 3:** writer comparison, LoRA tuning of the audio LM on ARC, robustness to imperfect response times. **Secondary, evaluation:** text metric and packaging the LM for offline inference.

Schedule

Dates	Milestones
Sep 22	Proposal. *Done.*
Oct 20	Checkpoint 1: pipeline built and tested on public audio. *Done.*
Oct 27	Data and resources released (moved from Oct 20). *Done:* data reader and statistics.
Oct 28 – Nov 10	Official-task pipeline, speech features, causal BEATs on GPU, grouped calibration, decode policies; ARC setup and dev caches; GPU training for Stage 2. **Checkpoint 2 (Nov 10).**
Nov 10 – 16	Training caches on ARC; full-scale Stage 2 training and calibration.
Nov 16 – Dec 2	Development leaderboard opens **Nov 16** (moved from Nov 4). Audio-LM writer vs. `rule`; semantic speech matching; container smoke test. **Team and resource declarations frozen Dec 2.**
Dec 3 – 11	Ablations. **Course final report Dec 11.**
Dec 12 – 23	Leaderboard closes **Dec 16**; **final container Dec 23.**
Dec 30	Rankings and paper invitations announced.